

Sesión 3 · Práctica 3.1 · Controles avanzados, componentes, validación y control de errores

Catálogo de máquinas de Contoso Coffee · práctica independiente

Dato	Valor
Duración	75 minutos (15 de ellos para montar el punto de partida)
Nivel	Intermedio
Requisitos previos	Ninguno: la práctica se construye desde cero
Archivos	DATOS/Machine-Order-Data.xlsx · DATOS/PedidosMaquinas_OneDrive.xlsx
Resultado	Catálogo con galerías enlazadas, selección múltiple en colección, pantalla de comparación, componente de cabecera reutilizable y formulario de pedido validado que escribe en una tabla de Excel

Escenario

Contoso Coffee quiere que sus empleados puedan pedir una cafetera para la sala de descanso desde el móvil o el navegador, comparar modelos y enviar el pedido. Vas a construir esa app: un catálogo de máquinas con una galería de tipos que filtra otra galería de modelos, casillas para seleccionar varias máquinas y compararlas, una cabecera convertida en componente reutilizable y un formulario de pedido que no deja enviar datos incorrectos y que gestiona los errores sin romperse.

Notación. Fórmulas con ; y ;; (navegador en español).

Parte 0 · Punto de partida

Esta parte monta el esqueleto de la app en unos quince minutos. Son pantallas y controles sueltos, sin lógica: todo lo que hace que la app funcione lo escribirás a partir de la Parte 1.

1. En <https://make.powerapps.com>, pulsa **Crear > Aplicación en blanco > Aplicación de lienzo en blanco**. Nombre: `Machine Ordering App`. Formato: **Tableta**.
2. Renombra `Screen1` como `Main Screen` y crea dos pantallas más en blanco: `Compare Screen` y `Order Screen`.
3. En cada una de las tres pantallas, inserta una **Etiqueta de texto** llamada `lblHeader` con estas propiedades. Es una cabecera provisional: en la Parte 5 la sustituiremos por un componente.

Propiedad	Valor
Text	"Machine Ordering" · "Comparar máquinas" · "Realizar pedido", según la pantalla
X · Y	0 · 0
Width · Height	Parent.Width · 60
Fill	RGBA(46; 20; 55; 1)
Color · Size	White · 20

1. En `Main Screen`, inserta un **Botón** `btnCompare` abajo a la derecha, con **Text** "Comparar" y **OnSelect** `Navigate('Compare Screen'; ScreenTransition.Cover)`.
2. En `Compare Screen`, inserta `btnBack` con **OnSelect** `Back()` y `btnOrder` con **Text** "Pedir esta máquina" y **OnSelect** `Navigate('Order Screen'; ScreenTransition.Fade)`.
3. En `Order Screen`, maqueta el formulario con controles sueltos y su etiqueta descriptiva al lado:

Control	Nombre	Ajuste
Entrada de texto	<code>txtQuantity</code>	Formato Número
Selector de fecha	<code>dteWanted</code>	—
Entrada de texto	<code>txtComments</code>	Multilínea, más alto
Botón	<code>btnSend</code>	Text "Enviar pedido"
Botón	<code>btnBackOrder</code>	Text "Volver", OnSelect <code>Back()</code>

1. Guarda la app.

Si ya tienes esta app montada de otra práctica del curso, ábrela y salta directamente a la Parte 1.

Parte 1 · Cargar los datos del catálogo

1. Abre la app `Machine Ordering App` en modo edición.
2. En **Datos > Agregar datos**, busca **Importar desde Excel** y selecciona `Machine-Order-Data.xlsx` de la carpeta `DATOS`.
3. Marca las tablas `Machines` y `MachineTypes` y pulsa **Conectar**. Quedarán incrustadas en la app como datos estáticos.

Si no encuentras «Importar desde Excel». Sube el archivo a OneDrive y conéctalo con el conector **Excel Online (Business)**. Funciona igual, con la ventaja de que los datos se pueden actualizar sin volver a publicar la app.

Imágenes. Las fotos de las máquinas se cargan desde una dirección de Microsoft. Si en tu red no se ven, la práctica funciona igualmente.

Parte 2 · Galerías enlazadas

1. En `Main Screen`, inserta una **Galería vertical** y llámala `galTypes`. Configura:

Propiedad	Valor
Items	<code>MachineTypes</code>
X · Y	<code>0 · 60</code>
Width · Height	<code>200 · Parent.Height - 60</code>
Diseño	<i>Solo imagen o Imagen y título</i>

1. En la plantilla, pon la imagen con **Image** `ThisItem.Photo` y una etiqueta con **Text** `ThisItem.Name`.
2. Resalta el tipo seleccionado cambiando el **Fill** de la etiqueta. Usamos el color literal y no `lblHeader.Fill` porque en la Parte 5 esa etiqueta desaparece:

```
If(ThisItem.IsSelected; ColorFade(RGBA(46; 20; 55; 1); 60%); Transparent)
```

1. Inserta una **Galería horizontal** `galMachines` a la derecha, con **Items** filtrados por el tipo seleccionado:

```
Filter(Machines; 'Machine Type ID' = galTypes.Selected.'Type ID')
```

1. En la plantilla de `galMachines`: imagen con `ThisItem.Photo`, título con `ThisItem.'Machine Name'` y subtítulo con `Text(ThisItem.Price; "#.##0 €")`. Pon **Wrap count** a `2` para ver dos filas.
2. Prueba con **Alt+click** sobre cada tipo y comprueba que la galería de máquinas cambia.

Parte 3 · Selección múltiple con una colección

1. En la plantilla de `galMachines`, inserta una **Casilla** (checkbox) `chkCompare` con **Text** `"Comparar"`.
2. Configura sus eventos para añadir o quitar la máquina de la colección `colCompare`:

Propiedad	Valor
OnCheck	<code>Collect(colCompare; ThisItem)</code>
OnUncheck	<code>Remove(colCompare; ThisItem)</code>
Default	<code>!IsBlank(LookUp(colCompare; 'Machine ID' = ThisItem.'Machine ID'))</code>

1. La propiedad **Default** hace que la casilla recuerde su estado aunque cambies de tipo en la galería de la izquierda. En la casilla moderna esta propiedad se llama **Checked**.
2. Modifica el botón `btnCompare` que creaste en la Parte 0:

Propiedad	Valor
Text	<code>"Comparar " & CountRows(colCompare) & " máquina(s)"</code>
DisplayMode	<code>If(CountRows(colCompare) >= 2; DisplayMode.Edit; DisplayMode.Disabled)</code>
OnSelect	<code>Navigate('Compare Screen'; ScreenTransition.Cover)</code>

1. Añade un botón `btnClear` con texto `"Limpiar selección"` y **OnSelect** `Clear(colCompare)`.

Parte 4 · Pantalla de comparación

1. En `Compare Screen`, inserta una **Galería horizontal** `galCompare` con **Items** `colCompare` y **Wrap count** `1`.
2. En la plantilla, muestra imagen, nombre, precio y estas características, cada una en una etiqueta:

Etiqueta	Text
Característica	<code>ThisItem.Feature</code>
Tazas por semana	<code>"Tazas/semana: " & ThisItem.'Avg. Cups/Week'</code>
Espressos por semana	<code>"Espressos/semana: " & ThisItem.'Avg. Espressos/Week'</code>
Descripción	<code>ThisItem.Description</code>

1. Resalta la máquina más barata con el **Fill** de la plantilla (**TemplateFill** de la galería):

```
If(ThisItem.Price = Min(colCompare; Price); RGBA(223; 246; 221; 1); White)
```

1. En **OnSelect** de la galería guarda la máquina elegida: `Set(gblSelectedMachine; ThisItem)`.
2. El botón `btnOrder` debe estar deshabilitado hasta que se elija una máquina: **DisplayMode** `If(IsBlank(gblSelectedMachine); DisplayMode.Disabled; DisplayMode.Edit)`.

Parte 5 · Componente de cabecera reutilizable

En la Parte 0 copiaste la misma cabecera en las tres pantallas. Si mañana cambias el color, tendrás que hacerlo tres veces. Un **componente** resuelve esto: se diseña una vez y se reutiliza en todas las pantallas, con propiedades de entrada que cambian lo necesario en cada una.

1. En la vista de árbol, cambia a la pestaña **Componentes** y pulsa **+ Nuevo componente**. Llámalo `cmpHeader`.
2. Ajusta su tamaño: **Width** `App.Width` y **Height** `60`.
3. Crea una propiedad personalizada: en el panel derecho, **+ Nueva propiedad personalizada**, nombre `Title`, tipo **Entrada**, tipo de datos **Texto**, valor predeterminado `"Machine Ordering"`.
4. Dentro del componente, inserta un rectángulo de fondo y dos etiquetas: una con **Text** `cmpHeader.Title` y otra con `"Hola, " & User().FullName`.
5. Crea una segunda propiedad de entrada `ShowBack` de tipo **Booleano** y un icono de flecha atrás con **Visible** `cmpHeader.ShowBack`.
6. Para que el icono pueda navegar, crea una propiedad de **comportamiento** `OnBack` y pon en **OnSelect** del icono `cmpHeader.OnBack()`.
7. Vuelve a **Pantallas**, borra la etiqueta `lblHeader` de cada pantalla e inserta el componente desde *Insertar > Componentes personalizados*. En cada pantalla ajusta:

Pantalla	Title	ShowBack	OnBack
Main Screen	<code>"Catálogo"</code>	<code>false</code>	vacío
Compare Screen	<code>"Comparar máquinas"</code>	<code>true</code>	<code>Back()</code>
Order Screen	<code>"Realizar pedido"</code>	<code>true</code>	<code>Back()</code>

Ir más allá. Si guardas el componente en una **biblioteca de componentes**, podrás usarlo en otras apps de la organización y, cuando lo actualices, todas las apps te avisarán para aplicar el cambio.

Parte 6 · Validación de datos en el formulario de pedido

1. En `Order Screen`, inserta una etiqueta `lblMachine` que muestre la máquina elegida y su precio: `gblSelectedMachine.'Machine Name' & " · " & Text(gblSelectedMachine.Price; "#.##0 €")`.
2. Ya tienes `txtQuantity`, `dteWanted` y `txtComments` de la Parte 0. Añade un cuadro `txtEmail` para el correo de contacto.
3. Selecciona el objeto **App** en la vista de árbol y, en su propiedad **Formulas**, define unas **fórmulas con nombre** con los parámetros de negocio del pedido:

```
fxMaxQty = 5;
fxMinDate = DateAdd(Today(); 3; TimeUnit.Days);
```

Fórmulas con nombre. Se escriben en `App.Formulas`, se recalculan solas y se pueden usar desde cualquier pantalla. Son preferibles a guardar valores en variables desde `App.OnStart`, porque nunca se quedan desactualizadas. Si mañana el máximo pasa a ser 10, solo lo cambias aquí. Cada definición termina en `;`.

1. Debajo de cada campo, inserta una etiqueta de error en rojo que solo se vea cuando la regla no se cumple y el usuario ya ha escrito algo:

Etiqueta	Visible	Text
lblErrQty	<code>!IsBlank(txtQuantity.Text) && ! (Value(txtQuantity.Text) >= 1 && Value(txtQuantity.Text) <= fxMaxQty)</code>	"Indica una cantidad entre 1 y " & fxMaxQty
lblErrDate	<code>!IsBlank(dteWanted.SelectedDate) && dteWanted.SelectedDate < fxMinDate</code>	"La fecha debe ser igual o posterior al " & Text(fxMinDate; "dd/mm/ yyyy")
lblErrEmail	<code>!IsBlank(txtEmail.Text) && ! IsMatch(txtEmail.Text; Match.Email)</code>	"El correo no tiene un formato válido"

1. Si usas controles modernos, marca también el borde en rojo con la propiedad **ValidationState** del cuadro de correo: `If(!IsBlank(Self.Text) && !IsMatch(Self.Text; Match.Email); ValidationState.Error; ValidationState.None)`.
2. Haz que `btnSend` solo se habilite cuando todo es correcto. Usa `With` para dar nombre a cada regla y que la fórmula se lea con facilidad:

```
With(
{
    qtyOk: Value(txtQuantity.Text) >= 1 && Value(txtQuantity.Text) <= fxMaxQty;
    dateOk: !IsBlank(dteWanted.SelectedDate) && dteWanted.SelectedDate >= fxMinDate;
    emailOk: IsMatch(txtEmail.Text; Match.Email)
};
If(qtyOk && dateOk && emailOk && !IsBlank(gblSelectedMachine); DisplayMode.Edit;
DisplayMode.Disabled)
)
```

Parte 7 · Control de errores

Aunque validemos en la app, la llamada al origen de datos puede fallar: se cae la red, alguien ha renombrado una columna o el archivo está bloqueado porque otra persona lo

tiene abierto. Con un Excel como origen esto pasa más de lo que parece, así que vamos a provocarlo y a gestionarlo.

1. Sube `DATOS/PedidosMaquinas_OneDrive.xlsx` a tu OneDrive para la Empresa y renómbralo con tus iniciales, por ejemplo `PedidosMaquinas_JL.xlsx`. Trae la tabla `Pedidos` con las columnas `Id`, `Title` (máquina), `Cantidad`, `FechaDeseada`, `Email` y `Comentarios`.
2. En la app, **Datos > Agregar datos > OneDrive para la Empresa**, elige el archivo y marca la tabla `Pedidos`.
3. Cambia el **OnSelect** de `btnSend` para guardar el pedido y gestionar el error con `IfError`:

```
IfError(
    Patch(Pedidos; Defaults(Pedidos);
        {
            Id: "PED-" & Text(Now(); "yyyymmddhhmmss");
            Title: gblSelectedMachine.'Machine Name';
            Cantidad: txtQuantity.Text;
            FechaDeseada: Text(dteWanted.SelectedDate; "yyyy-mm-dd");
            Email: txtEmail.Text;
            Comentarios: txtComments.Text
        });
    Notify("No se ha podido registrar el pedido: " & FirstError.Message;
NotificationType.Error);
    Notify("Pedido registrado correctamente"; NotificationType.Success);;
    Reset(txtQuantity);; Reset(txtEmail);; Reset(txtComments);;
    Navigate('Main Screen'; ScreenTransition.UnCover)
)
```

1. Fíjate en dos detalles del origen Excel: la fecha se guarda como texto `aaaa-mm-dd`, porque el conector no tiene tipos, y el identificador lo pone la app, porque Excel no numera las filas.
2. **Primer error a propósito: una columna que cambia de nombre.** Abre el archivo en Excel para la web y renombra la columna `Cantidad` como `Unidades`. Vuelve a la app, envía un pedido y comprueba que aparece la notificación de error en lugar de un fallo silencioso. Deshaz el cambio y vuelve a probar.
3. **Segundo error a propósito: el archivo bloqueado.** Abre el archivo con Excel de escritorio y déjalo abierto mientras envías otro pedido. Es el fallo más típico de este origen en una oficina: alguien deja el Excel abierto y la app deja de escribir.
4. Abre **Herramientas avanzadas > Monitor** (o *Supervisión* en el menú de la app), vuelve a enviar el pedido y localiza la llamada al conector de Excel que ha fallado y su mensaje de error. Monitor es la herramienta para diagnosticar qué pasa entre la app y los orígenes de datos.

Validar en la app no es opcional con Excel. Un origen de empresa puede rechazar un dato mal formado: obliga a que una columna tenga valor, comprueba que una fecha es una fecha y que un número es un número. Excel acepta cualquier cosa en cualquier celda. Todo lo que has hecho en la Parte 6 es, con este origen, la única barrera que existe.

Parte 8 · Usabilidad

1. Ejecuta el **Comprobador de aplicaciones** (icono de estetoscopio de la barra superior) y corrige al menos tres avisos de accesibilidad o rendimiento.
2. Revisa que el orden de tabulación del formulario es lógico (propiedad **TabIndex**).
3. Añade un indicador de carga: en `btnSend`, antes del `Patch`, `UpdateContext({ locSaving: true })`, y al final `UpdateContext({ locSaving: false })`; inserta un control **Spinner** con **Visible** `locSaving`.
4. Guarda y publica.

Comprobación

- La app tiene tres pantallas con navegación y un formulario de pedido maquetado.
- Al elegir un tipo, la galería de máquinas se filtra y el tipo queda resaltado.
- Se pueden marcar varias máquinas, el botón de comparar se habilita a partir de dos y la selección se mantiene al cambiar de tipo.
- La pantalla de comparación muestra las máquinas elegidas y resalta la más barata.
- Las tres pantallas usan el componente `cmpHeader` con título distinto y botón atrás donde corresponde.
- El formulario muestra mensajes de error por campo y no se puede enviar hasta que todo es válido.
- Un error del origen de datos se muestra al usuario con `IfError` y no rompe la app, tanto si falta una columna como si el archivo está bloqueado.